



MealSaver

Project Documentation

Rescuing today's meals, nourishing a better tomorrow.

Version 1.0.0	Document Type Full Product Documentation
Date May 2025	Platform Web Application

ABSTRACT

Food wastage is a growing global issue with significant environmental and economic impacts. To address this, the MealSaver application is developed to reduce food wastage by connecting surplus food donors — such as restaurants, cafes, and bakeries — with recipients in need, including NGOs, shelters, and community kitchens. The application facilitates seamless communication through real-time notifications and location-based matching, enhancing the redistribution of surplus food.

MealSaver is built as a full-stack web application using React and TypeScript for the frontend, Node.js with Express for the backend API, and PostgreSQL as the primary relational database. The system supports three distinct user roles: Donors, NGO Recipients, and Admins. Donors can upload food details including type, quantity, expiry time, and pickup location; NGOs receive real-time notifications and can accept nearby donations; Admins manage verifications, monitor system health, and handle emergency fallback logic.

Key features include a smart NGO-matching algorithm (match scoring up to 96%), OTP-based pickup verification, impact dashboards tracking meals saved and CO2 avoided, and a CSR-ready downloadable impact report. By initially focusing on local communities, MealSaver encourages sustainable food practices while fostering partnerships with environmental organizations. With scalability in mind through PostgreSQL and modular architecture, the platform has the potential to expand regionally and nationally.

TABLE OF CONTENTS

Sr. No.	Description
1	Chapter 1: Introduction
	1.1 Background
	1.2 Objectives
	1.3 Purpose, Scope, and Applicability
	1.4 Achievements
	1.5 Organisation of Report
2	Chapter 2: Survey of Technologies
3	Chapter 3: Requirements and Analysis
	3.1 Problem Definition
	3.2 Requirements Specification
	3.3 Planning and Scheduling (Gantt, PERT)
	3.4 Software and Hardware Requirements
	3.5 Preliminary Product Description
	3.6 Conceptual Models
4	Chapter 4: System Design
	4.1 Basic Modules
	4.2 Schema Design
	4.3 Procedural Design
	4.4 Security Issues
	4.5 Test Cases Design

LIST OF FIGURES

Sr. No.	Figure Name
01	App Overview – Food Redistribution Flow
02	Landing Page UI Mockup
03	Login & Registration Screen
04	Donor Registration Form
05	Donor Dashboard
06	Create Donation Form
07	Donation Status Detail Page
08	NGO Registration Form
09	Nearby Donations – NGO Dashboard
10	Donation Match Detail & Acceptance (96% Match Score)
11	Accepted Pickup Screen
12	Pickup Assignment Screen
13	OTP Pickup Verification
14	Delivery Confirmation Screen
15	Impact Report Dashboard
16	Admin Dashboard
17	Gantt Chart
18	PERT Chart
19	Data Flow Diagram (Level 0 & Level 1)
20	Use Case Diagram
21	Class Diagram
22	E-R Diagram
23	System Flow Diagram
24	PostgreSQL Schema Diagram
25	Process Flow Diagram

LIST OF TABLES

Sr. No.	Table Name
01	Gantt Chart Table
02	PERT Chart
03	Admin Attributes
04	Recipient / NGO Attributes
05	Donor Attributes
06	FoodItem / Donation Attributes
07	Software Requirements
08	Test Case Design

Chapter 1 – Introduction

In response to the global challenge of food wastage, food waste reduction applications have emerged as innovative solutions to connect surplus food with those in need. These apps aim to minimize food waste, promote sustainability, and support community well-being by efficiently redistributing excess food. One such initiative is MealSaver.

"MealSaver" embodies its mission to rescue meals from being wasted and ensure they reach people who can benefit from them. By facilitating the redistribution of surplus food between donors (restaurants, cafes, bakeries) and recipients (NGOs, shelters, community kitchens), MealSaver aims to conserve resources and reduce environmental impact, contributing to a more sustainable future.

1.1 Background

Food wastage is a significant global problem, with an estimated one-third of all food produced worldwide being lost or wasted annually. This waste occurs at various stages, including production, processing, retail, and consumption. The environmental impact of food waste is considerable, contributing to greenhouse gas emissions and unnecessary resource usage such as water and energy.

Key statistics highlighting the importance of addressing food waste:

- Approximately 1.3 billion tons of food is wasted globally each year.
- Food waste contributes to about 8% of global greenhouse gas emissions.
- Reducing food waste could save \$1 trillion annually.
- In India alone, food worth ₹92,000 crore is wasted every year.

MealSaver integrates real-time notifications, location-based NGO matching (within 5–7 km radius), OTP verification, and impact analytics to create a reliable and efficient system for reducing food waste and fostering a culture of sustainability.

1.2 Objectives

The MealSaver project aims to tackle the global issue of food wastage through the following key objectives:

1. Reduce food waste through efficient, technology-driven redistribution from donors to verified NGOs.
2. Promote sustainability by raising awareness and encouraging eco-friendly food practices.
3. Support community well-being by improving access to nutritious food for those in need.
4. Enhance operational efficiency with real-time notifications, location-based matching, and automated status updates.
5. Educate users about food wastage issues through in-app content and impact tracking.
6. Foster partnerships with local NGOs, shelters, and community kitchens.
7. Measure impact through data analytics — tracking meals saved, waste reduced, and CO2 avoided.
8. Enable scalability through modular architecture using React, Node.js, TypeScript, and PostgreSQL.

1.3 Purpose, Scope, and Applicability

1.3.1 Purpose

The purpose of MealSaver is to address the global challenge of food wastage by creating a digital platform that facilitates the efficient redistribution of surplus food. By connecting verified food donors with verified recipient NGOs, MealSaver aims to minimize food waste, promote sustainability, and support community well-being.

1.3.2 Scope

The scope of MealSaver encompasses the development and deployment of a full-stack web application. The platform allows individuals and businesses to donate surplus food items and enables NGOs facing food insecurity to access nutritious meals. The scope includes:

- Donor onboarding with food license verification.
- NGO registration with document verification and service radius configuration.
- Smart donation matching algorithm with 96% match score capability.
- Real-time activity timeline and status tracking for all donations.
- Impact reporting with meals saved, waste reduced, and CO2 impact metrics.
- Admin dashboard for managing verifications, emergency fallback, and platform oversight.
- CSR-ready downloadable impact reports for partner organizations.

1.3.3 Applicability

- Restaurants, Cafes, and Bakeries: Manage surplus inventory, reduce disposal costs, and build a socially responsible brand.
- NGOs, Shelters, and Community Kitchens: Access nearby surplus food donations through verified, time-bound pickup assignments.
- Environmental Organizations: Partner with MealSaver to promote eco-friendly practices and track CO2 impact.
- Corporate / CSR Teams: Download professional impact reports to demonstrate social responsibility.
- Admin / Platform Operators: Monitor user verifications, handle emergency cases, and ensure food safety compliance.

1.4 Achievements

Reduction in Food Waste

- Platform tracks quantifiable reduction: 1,240 meals saved, 320 kg waste reduced (as shown in impact dashboard).
- Real-time expiry-aware matching ensures food reaches recipients within safe usage windows.

User Engagement

- Three-role system (Donor, NGO/Recipient, Admin) with dedicated dashboards for each.
- Real-time notifications when donations are matched, accepted, picked up, or delivered.
- Activity timeline on every donation for complete transparency.

Technical Milestones

- Full-stack implementation with React + TypeScript frontend, Node.js + Express backend, PostgreSQL database.
- Smart NGO matching algorithm computing match scores based on distance, urgency, quantity fit, and storage compatibility.
- OTP-based pickup verification for food safety and trust.
- Emergency fallback logic: auto-notifies backup NGOs when no one accepts within time window.
- CSR/Sponsor-ready downloadable PDF impact report generation.

Scalability and Growth

- PostgreSQL relational database supports growth to large user bases and complex queries.
- Modular Node.js architecture allows adding new features (delivery partners, analytics APIs) without restructuring.
- Service radius configuration (5–7 km, adjustable) enables geographic expansion.

1.5 Organisation of Report

This project report is organized into four chapters:

- Chapter 1 – Introduction: Background, objectives, purpose, scope, achievements, and report structure.
- Chapter 2 – Survey of Technologies: Detailed discussion of all technologies used — React, TypeScript, Node.js, Express, PostgreSQL, and supporting libraries.
- Chapter 3 – Requirements and Analysis: Problem definition, requirement specification, planning (Gantt/PERT), hardware/software requirements, product description, and conceptual models (DFD, Use Case, Class, ER, System Flow diagrams).
- Chapter 4 – System Design: Basic modules, schema design, procedural design (logic, process, UI), security issues, and test cases.

Chapter 2 – Survey of Technologies

MealSaver is developed as a full-stack web application leveraging modern JavaScript/TypeScript technologies. The technology stack was chosen for its performance, developer experience, scalability, and strong ecosystem support. Below is a detailed survey of all technologies used.

2.1 Frontend Technologies

2.1.1 React (v18)

React is the primary frontend library used for building the MealSaver user interface. Developed and maintained by Meta, React uses a component-based architecture and a virtual DOM for efficient UI updates. In MealSaver, React is used to build all user-facing screens including the landing page, donor dashboard, NGO dashboard, donation forms, and impact reports.

- Component-based architecture for reusable UI elements (cards, forms, tables, dashboards).
- Virtual DOM ensures efficient re-rendering when donation status changes in real time.
- React Router DOM for client-side navigation between Donor, NGO, and Admin sections.
- React Hooks (useState, useEffect, useContext) for local state and side-effect management.

2.1.2 TypeScript

TypeScript is used throughout both the frontend and backend codebase. As a statically typed superset of JavaScript, TypeScript provides compile-time type checking, which reduces runtime errors and improves code maintainability.

- Type-safe API request/response interfaces between frontend and backend.
- Strongly typed props for all React components, preventing undefined behaviour.
- Enum types for donation status (Available, Accepted, Picked Up, Delivered, Expired).
- Interface definitions for User, Donor, NGO, Food Item, Donation, and Match Score models.

2.1.3 CSS / Tailwind CSS

The frontend styling uses a combination of Tailwind CSS utility classes and custom CSS to achieve the clean, green-themed Material-inspired design seen in the UI mockups. The color palette is centered on shades of green (#1B5E20, #388E3C) representing freshness and sustainability.

2.2 Backend Technologies

2.2.1 Node.js

Node.js serves as the backend runtime environment for MealSaver. Its non-blocking, event-driven I/O model is well-suited for handling concurrent API requests from multiple donors and NGOs simultaneously. Node.js is also used for running scheduled tasks such as expiry checks and automated fallback notifications.

- Handles concurrent HTTP requests from Donor, NGO, and Admin clients.
- Runs background workers for expiry-based donation status updates.
- Supports WebSocket or SSE for real-time activity timeline updates.

2.2.2 Express.js

Express.js is the web application framework used to build the RESTful API for MealSaver. It provides a minimal and flexible set of features for routing, middleware, and HTTP request handling.

- REST API routes for: /auth, /donors, /donations, /NGO, /admin, /impact.
- Middleware for JWT authentication, request validation, and error handling.
- Multer middleware for handling food license document and image uploads.
- CORS configuration for secure cross-origin requests from the React frontend.

2.3 Database Technology

2.3.1 PostgreSQL

PostgreSQL is used as the primary relational database for MealSaver. As a powerful, open-source object-relational database system, PostgreSQL provides ACID compliance, complex query support, and robust data integrity — essential for managing food donation records, user profiles, and impact metrics.

- Relational schema with foreign key constraints between Users, Donations, NGOs, and Matches.
- PostgreSQL ENUM types for donation status and food category fields.
- Full-text search capabilities for querying donations by food type or location.
- Indexing on geolocation fields (latitude, longitude) for fast nearby-NGO queries.
- Triggers for automated timestamp updates (created_at, updated_at).

2.3.2 Prisma ORM / pg. (node-Postgres)

Database interaction from the Node.js backend is managed through either Prisma ORM or the pg (node-postgres) library, providing type-safe database queries, connection pooling, and database migration management.

- Type-safe query builder prevents SQL injection and runtime type errors.
- Migration files track all schema changes from initial setup to current production state.
- Connection pooling handles multiple concurrent database connections efficiently.

2.4 Supporting Technologies

2.4.1 JWT (JSON Web Tokens)

JWT is used for stateless authentication across all three user roles (Donor, NGO, Admin). Upon login, the server issues a signed JWT containing the user's role and ID; this token is sent with every subsequent API request for authorization.

2.4.2 Bcrypt

Bcrypt is used for hashing user passwords before storing them in the PostgreSQL database. It ensures that even if the database is compromised, plaintext passwords cannot be recovered.

2.4.3 Multer / Cloudinary

Multer handles multipart form-data for file uploads (food license PDFs, donation images). Uploaded files are stored in Cloudinary (cloud storage) or a local file system, with the URL reference saved to PostgreSQL.

2.4.4 Nodemailer / Firebase Cloud Messaging

Real-time and email notifications are handled through Nodemailer for email alerts and Firebase Cloud Messaging (FCM) for push notifications — alerting NGOs of new nearby donations and donors when their donation is accepted or picked up.

2.4.5 Chart.js / Recharts

The Impact Report dashboard (showing Meals Saved vs. Waste Reduced over time) is built using Chart.js or Recharts, rendering interactive line charts with month-over-month growth data.

Chapter 3 – Requirements and Analysis

3.1 Problem Definition

The MealSaver application addresses the critical issue of food wastage, which has far-reaching economic, environmental, and social consequences. Despite various existing tools, there is a significant gap in comprehensive, real-time solutions that connect food donors directly with verified recipient organizations in a time-sensitive, location-aware manner.

Key challenges identified during development:

9. Real-Time Matching Complexity: Surplus food has a very short viable window (often 2–6 hours). The system must match donors with the nearest available verified NGO within minutes, not hours.
10. Multi-Role User Management: Managing three distinct roles (Donor, NGO/Recipient, Admin) with different dashboards, permissions, and workflows within a single cohesive application.
11. Data Integrity and Food Safety: Ensuring donation details (expiry times, food type, handling instructions) are accurately recorded and that only verified NGOs can accept donations.
12. Scalability of PostgreSQL Schema: Designing a relational schema that handles growing users, donations, and matches without performance degradation.
13. Asynchronous Operations: Background tasks such as expiry monitoring, fallback NGO notification, and impact metric aggregation must run without blocking the main API thread.

3.2 Requirements Specification

System Entities

The MealSaver system has three primary entities:

- Donor: Restaurants, cafes, bakeries, and individuals with surplus food.
- Recipient (NGO): Verified NGOs, shelters, community kitchens, and orphanages.
- Admin: Platform administrators managing verifications, disputes, and system health.

Functional Requirements

Authentication & User Management:

- Register, login, and profile management for all three roles.
- Food license / organization document verification for Donors and NGOs.
- JWT-based secure session management.

Donor Functionality:

- Create donation with food title, category, type (Cooked/Raw/Packaged/Bakery), quantity, unit, veg/non-veg flag, preparation time, expiry/safe usage time, pickup address, preferred pickup window, special instructions, and up to 5 food images.
- View active donations with real-time status (Available → NGOs Notified → Accepted → Pickup Assigned → Picked Up → Delivered).
- Receive push/email notifications when donation is accepted or picked up.
- View and download impact report (meals saved, waste reduced, CO2 avoided, total value contributed).

NGO / Recipient Functionality:

- Browse nearby donations within configurable service radius (default 5–7 km).
- View donation match detail with match score, distance, urgency alignment, quantity fit, and storage compatibility.

- Accept or reject donation; accepted donations are reserved and removed from other NGOs' available lists.
- Perform OTP-based pickup verification.
- Confirm delivery with receiver note.
- Track delivery history and impact metrics.

Admin Functionality:

- Dashboard overview: pending verifications, active donations, expiring soon, emergency cases.
- Verify Donor and NGO accounts (approve/reject document uploads).
- Manual matching for unassigned donations.
- Emergency handling: auto-notify backup NGOs if no acceptance within time window.
- View and manage all users, donations, and system alerts.

Non-Functional Requirements

- Reliability: PostgreSQL ACID transactions ensure no donation data is lost.
- Availability: Application should be available 24/7 with auto-restart on failure.
- Security: JWT authentication, bcrypt password hashing, input validation/sanitization, HTTPS.
- Performance: Geolocation-indexed PostgreSQL queries for NGO matching must respond within 200ms.
- Scalability: Modular Node.js architecture supports horizontal scaling via load balancers.
- Maintainability: TypeScript throughout ensures type-safe, self-documenting codebase.

3.3 Planning and Scheduling

3.3.1 Gantt Chart

Phase	Start Date	End Date	Days	Jul	Sep–Dec	Jan–Feb
Planning	05-Jul-2024	10-Jul-2024	5			
Research & Analysis	11-Jul-2024	14-Aug-2024	32			
Design	14-Aug-2024	21-Sep-2024	37			
Coding	21-Sep-2024	21-Jan-2025	121			
Testing	21-Jan-2025	21-Feb-2025	30			
Evaluation & Error Handling	21-Jan-2025	23-Feb-2025	30			
Report Submission	23-Feb-2025	28-Feb-2025	5			

3.3.2 PERT Chart (Activity Network)

The PERT (Program Evaluation and Review Technique) chart for MealSaver defines the critical path: Planning → Research & Analysis → Design → Coding → Testing → Evaluation & Error Handling → Report Submission. The critical path duration is approximately 239 days from project start (5-Jul-2024) to submission (28-Feb-2025). The Coding phase (121 days) is the longest activity and lies on the critical path.

3.4 Hardware and Software Requirements

Hardware Requirements – Development Machine

Component	Minimum Requirement	Used for Development
Processor	Intel Core i5 or AMD equivalent	Intel Core i3
RAM	8 GB	8 GB
Storage	SSD with 250 GB free	SSD with 512 GB
Display	1920 x 1080 resolution	1920 x 1080 resolution
Operating System	Windows 10 / macOS 10.14+ / Ubuntu 20.04	Windows 11
Browser	Chrome 90+ / Firefox 88+	Chrome (latest)

Software Requirements

Layer	Technology	Version / Tool
Frontend	React + TypeScript	React 18, TypeScript 5.x
Styling	Tailwind CSS / CSS Modules	Tailwind CSS v3
Routing	React Router DOM	v6
Backend	Node.js + Express.js	Node.js 18 LTS, Express 4.x
Language	TypeScript (backend)	TypeScript 5.x, ts-node
Database	PostgreSQL	PostgreSQL 15
ORM	Prisma ORM / node-postgres (pg)	Prisma 5.x
Auth	JWT + Bcrypt	jsonwebtoken, bcryptjs
File Upload	Multer + Cloudinary	multer v1.x
IDE	Visual Studio Code	VS Code (latest)
Version Control	Git + GitHub	git 2.x, GitHub
Package Manager	npm	npm 9.x
DB Client	pgAdmin / DBeaver	pgAdmin 4
Charts	Chart.js / Recharts	Recharts 2.x

3.5 Preliminary Product Description

MealSaver is a three-role platform. Below is the feature breakdown per role:

Admin	Donor	NGO / Recipient
Manage user verifications and profiles	Register and get verified with food license	Register with organization documents
Moderate donation postings	Create donations with full food details	Browse nearby donations (5–7 km radius)
Handle disputes and emergency cases	View real-time donation status timeline	View match score and food details

Generate platform-wide reports	Receive notifications on acceptance/pickup	Accept or reject donations
Manual matching for unassigned donations	View and download impact report	Perform OTP pickup verification
Monitor system health and alerts	Manage donation history and schedules	Confirm delivery with receiver notes

3.6 Conceptual Models

3.6.1 Data Flow Diagram (DFD)

Level 0 (Context Diagram): The MealSaver App sits at the center, interacting with four management subsystems:

- Login Management – handles authentication for all three roles.
- User Management – handles profile creation, verification, and updates.
- Food / Donation Management – handles donation creation, matching, and status updates.
- API Management – handles all data exchanges between frontend and PostgreSQL backend.

Level 1 (Detailed Flow): The USER sends requests to VIEW APPLICATION, SEARCH FOOD, LOGIN, ORDER/CLAIM FOOD, and LOGOUT. Each process interacts with the corresponding data store: App Data, Product/Food Data, User Data, and Order/Donation Data.

3.6.2 Use Case Diagram

Actors: DONOR, RECEIVER (NGO), ADMIN, TPV (Third-Party Verifier). Key use cases include: Register/Login, Place Food to Donate, View Donation Request, Accept Food Request, Track Donor/Receiver Location, Update Database, Notify User, Delivery of Food, Organic Compost (expired food fallback).

3.6.3 Class Diagram

Primary classes: Admin (adminID, username, email, password, accessLevel; methods: manageDonors(), manageRecipients(), manageFoodItems(), generateReports()), Donor (donorID, name, email, password, foodItems, recipes; methods: register(), login(), addFoodItem(), addRecipe()), Recipient (recipientID, name, email, password, claimedItems, suggestedRecipes; methods: register(), login(), viewAvailableFoodItems(), claimFoodItem()), FoodItem (itemID, foodName, quantity, expirationDate, pickupLocation; methods: addFoodItem(), updateFoodItem(), deleteFoodItem()), Recipe (recipeID, recipeName, description, ingredients).

3.6.4 E-R Diagram

Entity-Relationship model for MealSaver PostgreSQL database. Key entities: User (UserID, Username, Email, Password, MobileNumber, Role), FoodDetails (FoodID, Title, Category, Quantity, ExpiryDate, PickupLocation), NGO_List (NGOID, Name, Address, Contact, ServiceRadius), Donation (DonationID, DonorID→User, NGOID→NGO_List, FoodID→FoodDetails, Status, CreatedAt), Login (LoginID, UserID→User, Password). Relationships: User CAN Login, User interacts with FoodDetails through cart operations (Add, Remove, View, Proceed for Order), User interacts with NGO through Send Food NGO.

3.6.5 System Flow Diagram

New User registers as Donor, NGO/Recipient, or Admin → Login → Donor uploads food details (type, location, quantity, time/date) → Admin verifies and publishes donation → NGO/Recipient browses nearby donations, accepts, performs OTP pickup verification → Delivery confirmed → Feedback submitted → Admin updates/manages data → Logout.

Chapter 4 – System Design

4.1 Basic Modules

1. User Management Module

Handles user registration, login, profile management, and role-based access control. Includes the User entity with attributes: id, username, email, passwordHash, role (Donor/NGO/Admin), verificationStatus, createdAt. Methods: register(), login(), updateProfile(), verifyAccount().

2. Donation Management Module

Handles food donations created by donors. Includes the Donation entity (id, donorId, foodTitle, category, food Type, quantity, unit, veg NonVeg, preparation Time, expiry Time, pickup Address, preferred Window, instructions, imageUrls[], status, createdAt) and associated FoodItem class. The Donor class includes methods: addDonation(), viewDonations(), updateDonation(), cancelDonation().

3. NGO Matching Module

The core algorithmic module. Computes a match score (0–100%) for each nearby NGO based on: distance from donor (weighted 30%), urgency alignment — how well NGO pickup capacity aligns with the food's expiry window (25%), quantity fit — whether NGO storage can accommodate the donation quantity (25%), and storage compatibility — refrigerated/dry/ambient alignment (20%). Includes the FoodMatch entity: id, donationId, ngold, matchScore, status.

4. Request/Acceptance Management Module

Handles food claim requests made by NGOs. Once an NGO accepts a donation, the donation is locked (status: Accepted) and other NGOs see it as unavailable. Includes Pickup Assignment with: donationId, ngold, pickupMethod (NGO Self Pickup / Delivery Partner), assignedRider, trackingId, pickupWindow, status. Methods: acceptDonation(), rejectDonation(), assignPickup().

5. OTP Verification Module

Handles the OTP-based pickup verification step. When a pickup is initiated, a 4-digit OTP is generated server-side and sent to the recipient. The pickup agent enters the OTP on the app to confirm physical receipt. Methods: generateOTP(), verifyOTP(), confirmPickup().

6. Notification Module

Handles all real-time and push notifications. Includes: push notifications via Firebase Cloud Messaging (FCM) for mobile/web, email notifications via Nodemailer, and in-app activity timeline updates via WebSocket/SSE. Notification entity: id, userId, message, type (Donation Matched / Accepted / Picked Up / Delivered / Expired), isRead, createdAt.

7. Impact Tracking Module

Aggregates and displays donation impact metrics per donor and platform-wide. Tracks: meal saved (count), wasteReducedKg, co2ImpactKg (equivalent to car travel avoided), total Donations, totalValueContributed (INR), beneficiary NGOs. Generates CSR-ready downloadable PDF impact reports.

8. Admin Management Module

Platform administration including: user verification queue, donation moderation, manual matching for unassigned donations, emergency fallback logic (auto-notify backup NGOs within time window), and system health monitoring (alert badges for urgent cases).

4.2 Schema Design

PostgreSQL Database Schema

The following tables constitute the core PostgreSQL schema for MealSaver:

Column	Type	Constraints	Description
— TABLE: users —			
id	SERIAL	PK	Auto-increment primary key
username	VARCHAR(50)	NOT NULL	Display name
email	VARCHAR(100)	UNIQUE, NOT NULL	Login identifier
password_hash	VARCHAR(255)	NOT NULL	Bcrypt hashed password
role	ENUM('donor','ngo','admin')	NOT NULL	User role
is_verified	BOOLEAN	DEFAULT false	Admin verification status
created_at	TIMESTAMP	DEFAULT NOW()	Account creation time
— TABLE: donors —			
id	SERIAL	PK	Auto-increment primary key
user_id	INT	FK → users.id	References users table
business_name	VARCHAR(100)	NOT NULL	Restaurant/Cafe/Bakery name
business_type	VARCHAR(50)	NOT NULL	Restaurant/Cafe/Bakery/Other
phone	VARCHAR(15)	NOT NULL	Contact number
address	TEXT	NOT NULL	Business address
food_license_url	TEXT		FSSAI/food license document URL
latitude	DECIMAL(10,8)	NOT NULL	Geolocation (indexed)
longitude	DECIMAL(11,8)	NOT NULL	Geolocation (indexed)
— TABLE: ngos —			
id	SERIAL	PK	Auto-increment primary key

user_id	INT	FK → users.id	References users table
org_name	VARCHAR(100)	NOT NULL	NGO/Shelter name
service_radius_km	INT	DEFAULT 7	Configurable pickup radius
storage_capability	VARCHAR(50)	NOT NULL	Refrigerated/Dry/Ambient
— TABLE: donations —			
id	SERIAL	PK	Auto-increment primary key
donation_code	VARCHAR(20)	UNIQUE, NOT NULL	e.g. DNTN-2025-0524-0012
donor_id	INT	FK → donors.id	Donor reference
food_title	VARCHAR(100)	NOT NULL	e.g. Veg Biryani + Dal Tadka
category	VARCHAR(50)	NOT NULL	Main Course/Bakery/Raw Material
food_type	ENUM	NOT NULL	Cooked/Bakery/Packaged/Raw
quantity	INT	NOT NULL	Number of portions/kg/packages
unit	VARCHAR(20)	NOT NULL	Portions/Kg/Packages/Litres
is_veg	BOOLEAN	NOT NULL	Veg or Non-Veg
expiry_time	TIMESTAMP	NOT NULL	Safe usage deadline
pickup_addresses	TEXT	NOT NULL	Full pickup address
status	ENUM	DEFAULT 'available'	available/ngos_notified/accepted/pickup_assigned/picked_up/delivered/expired
image_urls	TEXT[]		Array of food image URLs
created_at	TIMESTAMP	DEFAULT NOW()	Submission timestamp

4.3 Procedural Design

4.3.1 Logic Flow Diagram

The donation processing logic follows these steps:

14. User Input: Donor submits food details via the Create Donation form.
15. Input Validation: System validates required fields, checks for duplicate active donations from same donor, verifies expiry time is in the future.
16. Database Insert: Valid donation is inserted into the donations table with status 'available'.
17. NGO Matching Algorithm: System queries NGOs table filtered by $\text{service_radius_km} \geq \text{distance to donor}$. For each candidate NGO, computes match score. Selects top-N NGOs sorted by score.
18. Notification: Push notifications sent to top-N NGOs via FCM. Status updated to 'NGOs notified'.
19. NGO Accepts/Rejects: First NGO to accept locks the donation (status $\rightarrow$ 'accepted'). System sends confirmation to donor and creates pickup_assignment record.
20. Pickup Verification: OTP generated, sent to NGO/receiver. Agent enters OTP $\rightarrow$ status $\rightarrow$ 'picked_up'.
21. Delivery Confirmation: NGO coordinator confirms receipt $\rightarrow$ status $\rightarrow$ 'delivered'. Impact metrics updated.
22. Expiry Monitoring (background): Cron job checks donations where $\text{expiry_time} < \text{NOW}()$ AND status $\neq$ 'delivered'. Marks as 'expired'. Notifies donor.

4.3.2 Process Flow Diagram

Start $\rightarrow$ User visits MealSaver $\rightarrow$ [Register? $\rightarrow$ Yes $\rightarrow$ Fill Form $\rightarrow$ Verify Documents $\rightarrow$ Account Created] OR [Login? $\rightarrow$ Yes $\rightarrow$ Enter Email + Password $\rightarrow$ JWT Issued]. After authentication: Donor $\rightarrow$ Create Donation $\rightarrow$ System Matches NGO $\rightarrow$ NGO Accepts $\rightarrow$ Pickup Verification (OTP) $\rightarrow$ Delivery Confirmed $\rightarrow$ Impact Updated. Recipient (NGO) $\rightarrow$ View Nearby Donations $\rightarrow$ View Match Score $\rightarrow$ Accept/Reject $\rightarrow$ Pickup $\rightarrow$ Deliver. Admin $\rightarrow$ Dashboard $\rightarrow$ Verify Users $\rightarrow$ Monitor Donations $\rightarrow$ Handle Emergency. All paths $\rightarrow$ Logout $\rightarrow$ End.

4.3.3 User Interface Design

The MealSaver UI is designed around four core screen categories as shown in the UI mockup PDF:

- Page 1 – Brand & Authentication: Landing page with impact stats (1,240 meals saved, 320 kg waste reduced, 72 Active NGOs), Login screen with role selector (Donor / NGO / Admin), Registration choice screen.
- Page 2 – Donor Onboarding & Donation Upload: Donor registration form, Donor dashboard with active donations and impact metrics, Create donation form with image upload, Donation status detail with activity timeline.
- Page 3 – NGO Flow, Matching & Acceptance: NGO registration form, Nearby donations dashboard with distance-sorted list, Donation match detail page with 96% match score breakdown, Accepted pickup confirmation screen.
- Page 4 – Pickup, Delivery, Admin & Impact: Pickup assignment screen, OTP verification screen, Delivery confirmation screen, Impact report with charts and CSR report download, Admin dashboard with verification queues and emergency handling.

4.4 Security Issues

The following security measures are implemented in MealSaver:

23. Data Encryption in Transit: All HTTP communication uses HTTPS/TLS. JWT tokens are transmitted in Authorization headers (Bearer scheme).
24. Password Security: User passwords are hashed using bcrypt with a salt factor of 12 before storage in PostgreSQL. Plaintext passwords are never stored.
25. Authentication and Authorization: JWT-based stateless authentication. Role-based access control (RBAC) middleware ensures Donors cannot access NGO routes and vice versa. Admin routes require explicit admin role verification.

26. Input Validation and Sanitization: All API inputs are validated using express-validator or Zod schemas. Parameterized SQL queries (via Prisma ORM / pg. prepared statements) prevent SQL injection attacks.
27. Secure File Upload: Uploaded files (food licenses, donation images) are validated for file type (PDF/JPG/PNG) and size (max 5MB) on the server side before processing.
28. Document Verification: Donor food licenses (FSSAI/PDF) and NGO registration documents are manually reviewed by Admin before account verification — preventing fraudulent donors/NGOs.
29. OTP Security: Pickup OTPs are time-limited (valid for 10 minutes), server-generated (not client-side), and single-use. OTP is invalidated immediately upon successful verification.
30. Rate Limiting: API rate limiting (express-rate-limit) prevents brute force login attacks and API abuse.

4.5 Test Case Design

Form Name	Test Case	Test Data	State	Input Value	Expected Result
Donor Register	Password & Confirm must match; Email must be unique; Phone 10 digits	Alphanumeric password, valid email, 10-digit phone	Valid	greenleafcafe@gmail.com / *****	Donor account created; redirected to dashboard
Donor Login	Email and password must be correct	Email must contain '@'	Valid	greenleafcafe@gmail.com	JWT issued; redirected to Donor Dashboard
Create Donation	Expiry time must be in future; Quantity > 0; Pickup address required	Food title, qty=8, unit=Portions, expiry=future	Valid	Veg Biryani, 8 Portions, 9:00 PM	Donation created; NGOs notified; status=available
NGO Register	Org name required; service radius valid; documents uploaded	Org name, valid email, radius=7, docs uploaded	Valid	Helping Hands NGO	NGO account created; pending admin verification
NGO Login	Email and password correct; account must be verified	Valid email and password	Valid	ngo@gmail.com	JWT issued; redirected to NGO Nearby Donations
Accept Donation	Donation must be available (not already accepted)	Donation status=available	Valid	DON-2024-0512	Donation locked; other NGOs see unavailable; pickup assigned
OTP Verification	OTP must be 4 digits; must match server-generated OTP; valid within 10 mins	4-digit numeric OTP	Valid	7 3 1 9	Pickup confirmed; donation status=picked_up
Admin Login	Email and password correct; role must be admin	Admin email and password	Valid	admin@mealsaver.org	Redirected to Admin Dashboard with full management access

Impact Report	Report must aggregate all delivered donations for the user	User has at least 1 delivered donation	Valid	View Impact Report	Correct totals for meals saved, waste reduced, CO2 avoided, value
---------------	--	--	-------	--------------------	---

5. Future Roadmap

5.1 Short-Term (0–3 Months)

- SMS and WhatsApp notifications for NGOs without smartphones
- Bulk donation upload via CSV for large institutional donors
- In-app direct messaging between donors and NGOs
- Google Maps turn-by-turn navigation integration for riders

5.2 Medium-Term (3–9 Months)

- Mobile app (Android & iOS) for NGOs and delivery partners
- Ratings and review system: NGOs can rate donors and vice versa
- Subscription tier for corporate donors with automated scheduling
- Multi-language support (Hindi, Kannada, Tamil)
- Inventory prediction model: alert donors when surplus is likely based on order patterns

5.3 Long-Term (9+ Months)

- API integration with POS systems (e.g., Pet Pooja, Posits) for automated surplus detection
- Government partnership portal for municipal food banks
- Carbon credit issuance integration for CSR donors
- Expansion to Tier-2 and Tier-3 cities with on-ground NGO onboarding programs
- B2B enterprise dashboard for large food companies managing multiple outlets

References

- ❖ React Documentation – <https://react.dev>
- ❖ TypeScript Handbook – <https://www.typescriptlang.org/docs/>
- ❖ Node.js Documentation – <https://nodejs.org/en/docs>
- ❖ Express.js Guide – <https://expressjs.com>
- ❖ PostgreSQL Documentation – <https://www.postgresql.org/docs/>
- ❖ Prisma ORM Documentation – <https://www.prisma.io/docs>
- ❖ JSON Web Tokens (JWT) – <https://jwt.io/introduction>
- ❖ Firebase Cloud Messaging – <https://firebase.google.com/docs/cloud-messaging>
- ❖ Recharts Library – <https://recharts.org>
- ❖ Tailwind CSS – <https://tailwindcss.com/docs>
- ❖ FAO – Food Wastage Footprint: Impacts on Natural Resources, 2013
- ❖ UNEP – Food Waste Index Report 2021